CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)
CSPIT — Department of Computer Engineering · ITUE203 Web Development Frameworks

Student Name	Dhruv Bhoi	**Student ID**	**D26CE154**
Academic Year	2026–27 (ODD)	**Semester**	3rd Semester, B.Tech CE

Practical 12: Event Management CRUD with Poster Upload and File Validation

Practical Aim & Tools

Aim: Build an Event Management CRUD interface with poster image upload, MIME type and size validation, secure unique filename generation, directory permissions, and database tracking.

Tools & Technologies: PHP File Upload Handling (`$_FILES`), MIME Type Detection (`finfo`), File Security

1. Secure File Upload Architecture (`manage_events.php`)

File uploading is one of the most common web attack vectors (Remote Code Execution, Malicious Script Injection). StudentHub defends poster uploads through 5 strict verification checks:
1. HTTP POST & Error Check: Verifies `is_uploaded_file()` and checks `$_FILES['poster']['error'] === UPLOAD_ERR_OK`.
2. File Size Cap: Restricts files to a strict 2 Megabyte ceiling (`2 * 1024 * 1024` bytes).
3. MIME Content Inspection: Uses PHP's `finfo(FILEINFO_MIME_TYPE)` on temporary file bytes, permitting only `image/jpeg`, `image/png`, and `image/webp`. Never relies on client-supplied extensions.
4. Cryptographic Renaming: Files are renamed using `bin2hex(random_bytes(16)) . '.' . $ext` to prevent path traversal and execution.
5. Directory Execution Prevention: The `uploads/` folder contains an `index.php` and an `.htaccess` disabling script execution.

2. Implementation Code: Poster Upload & MIME Inspection

Listing: manage_events.php — MIME Type Verification and Cryptographic Renaming

```
// manage_events.php - Poster Validation
if (!empty($_FILES['poster']['name'])) {
    $f = $_FILES['poster'];
    if ($f['size'] > 2 * 1024 * 1024) $errors['poster'] = 'Poster must be under 2 MB.';
    else {
        $finfo = new finfo(FILEINFO_MIME_TYPE);
        $mime = $finfo->file($f['tmp_name']);
        $map = ['image/jpeg'=>'jpg', 'image/png'=>'png', 'image/webp'=>'webp'];
        if (!isset($map[$mime])) {
```

```
            $errors['poster'] = 'Invalid format. Only JPG, PNG, and WEBP allowed.';
        } else {
            $filename = bin2hex(random_bytes(16)) . '.' . $map[$mime];
            move_uploaded_file($f['tmp_name'], BASE_PATH . '/uploads/' . $filename);
            $posterPath = 'uploads/' . $filename;
        }
    }
}
```

3. Verification Screenshot: Event Management CRUD

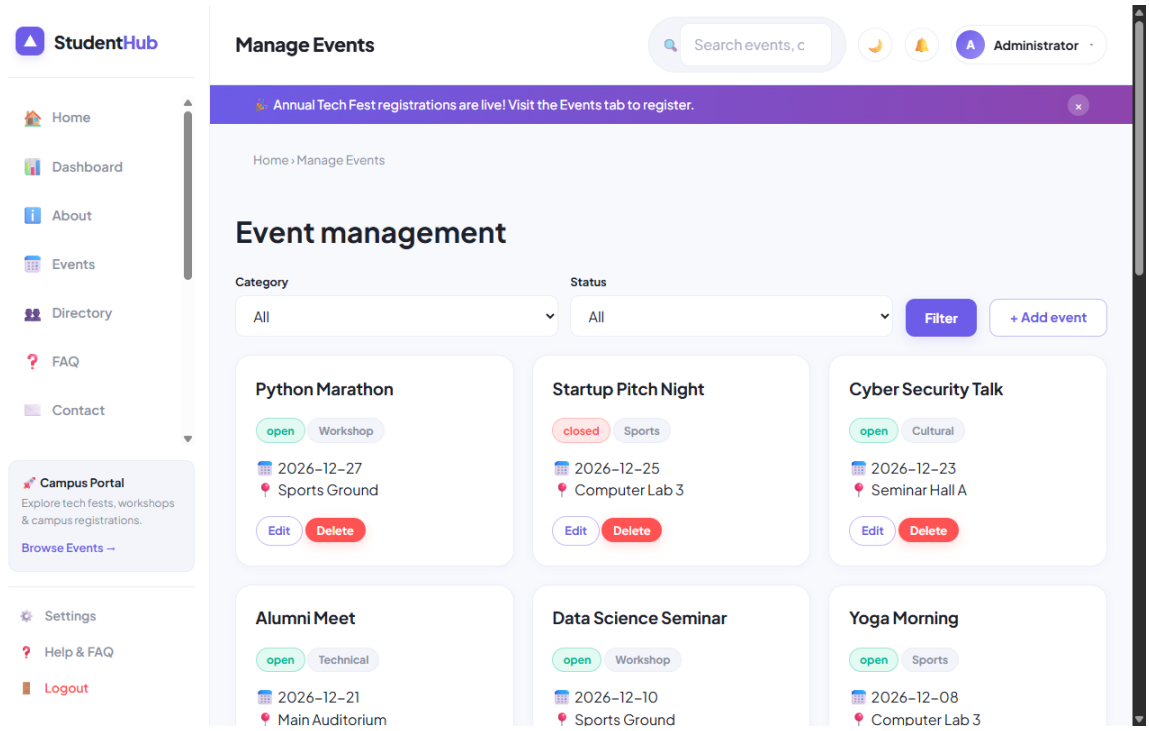


Figure: Event Management CRUD Interface with Poster Upload Field and Live Status Controls

4. Key Questions & Analysis

Q: Why should `$_FILES['poster']['type']` never be trusted?

Ans: `$_FILES['...']['type']` is a client-provided HTTP header that can be spoofed trivially using Postman or Burp Suite. A malicious user could upload a PHP shell named `shell.php` with header `Content-Type: image/jpeg`. Server-side verification must use `finfo_file` to read file magic bytes directly.

Q: How does cryptographic file renaming mitigate path traversal?

Ans: Renaming files to random hashes (e.g. `a3f8c...jpg`) strips dangerous characters like `../` or null bytes, preventing attackers from overwriting existing system files or guessing uploaded filenames.

Practical 12 Conclusion & Learning Outcomes

Built an Event Management administrative CRUD interface with secure file upload handling. Implemented MIME magic-byte verification (finfo), 2MB file size restrictions, cryptographic filename randomization, and path traversal mitigations.